HOW IT WORKS (COMPACT ARCHITECTURE)

Components

- Capture: Recorder + RecordBuffer
- Orchestration: Whisper (threaded task loop, chunk handling)
- Inference: Recognizer + ONNX Runtime + TensorUtils
- Long audio: AudioChunker + ChunkMerger
- Universal model support classes present: ModelManager, ModelConfig, AudioFeatureExtractor, BPETokenizer

Data flow

1. UI/service starts Recorder.
2. Recorder captures PCM audio (optional VAD) and stores bytes in RecordBuffer.
3. Whisper reads normalized float samples from RecordBuffer.
4. Long audio is split into 30-second chunks and merged after per-chunk inferenc
e.
5. Whisper invokes Recognizer with language/action.
6. Recognizer executes initializer -> encoder -> cache init -> decoder loop -> detokenizer and returns text/language via callbacks.
7. Caller displays text, commits text in IME, or returns recognition results.

Current engine status

- Dynamic/universal classes are present in repo.
- Active load path in Whisper currently uses legacy Recognizer constructor and helper ONNX files (initializer/cache initializer/detokenizer).
- ModelManager contains TODO for model preference selection wiring.

HOW TO RUN (MINIMAL)

1. Build/install Android app module :app.
2. Launch app; setup opens if model files are missing.
3. Download model ZIP and install via SetupActivity.
4. Use main record button, or enable ToneTunner IME / voice input service.

NOT FOUND IN REPO

- Formal benchmark results (latency/WER/device matrix).
- Explicit automated ASR test suite.
- End-to-end production runbook beyond in-app setup flow.